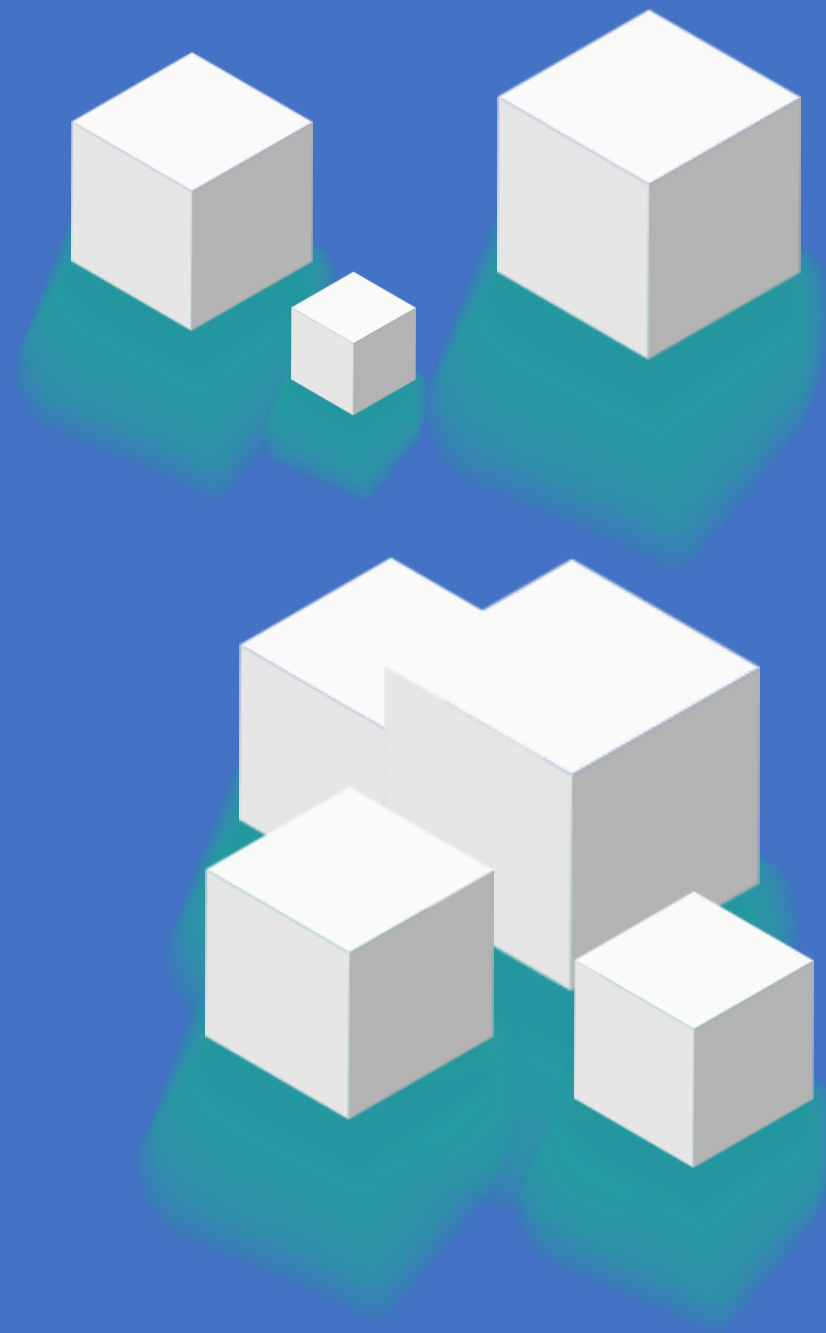
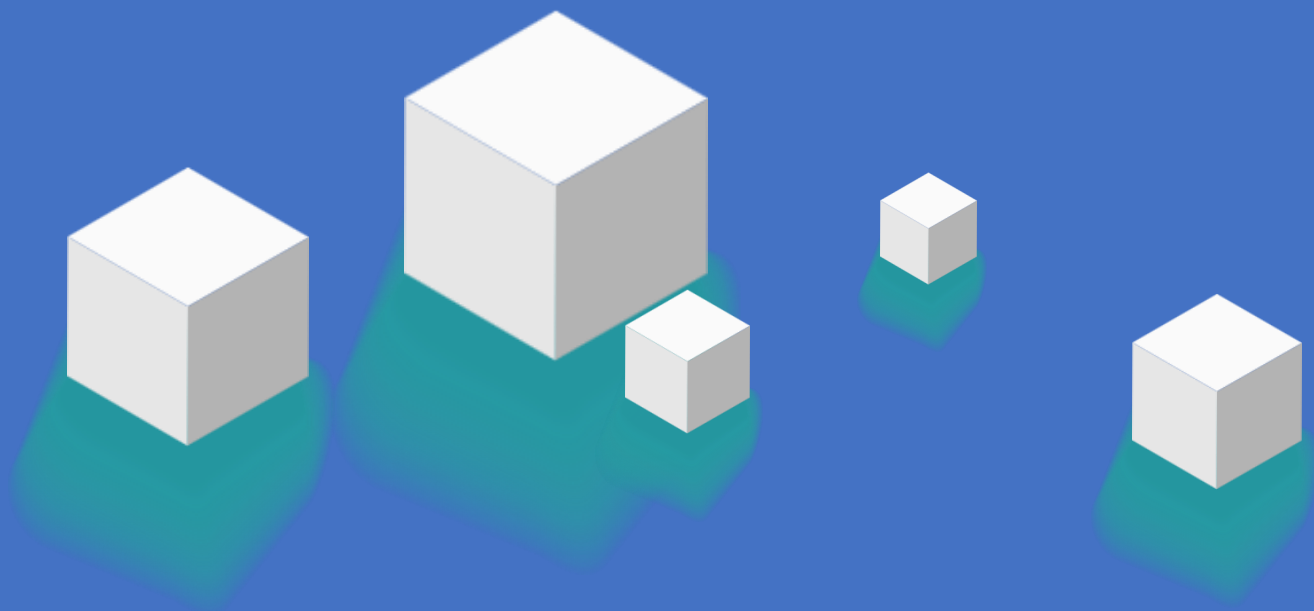


实盘作战与CEO控制台



>> 今天的学习目标

实盘作战与CEO控制台

- 实盘作战场景

工程原则、项目结构

- 自动闭环与分级告警

自动闭环、State 设计

写入的原子性、谁读谁写

主循环代码骨架

告警 4 级路由、定时调度

如何上生产

- CEO控制台

技术选型、控制台Tab、双向通信原理

- CASE：龙头战法

龙头战法 V1版本

龙头战法 V2版本

实盘原理

实盘作战场景

Thinking: 之前我们的工作范式是 按时机触发的工作流: 盘前 9:00 跑一次晨会 => 得到晨报。

现在我们想要持续的运行, 即 9:30 到 15:55 之间, 每分钟跑一次主循环, 全自动盯盘 + 下单 + 推送。

怎么让一个量化系统在人不盯的情况下安全、稳定地跑实盘?

怎么让 你 (CEO) 在不接管系统的前提下, 依然看得见 + 控得住?

=> TO DO: 打造 AI 团队, 7×24 自动跑实盘

实盘作战场景

AI量化系统 包括3个核心能力:

1) 自动闭环

位置: live_trading/

作用: 每分钟一次: 拉行情 → 评估信号 → 风控 → 下单 → 推送

2) CEO 控制台

位置: app.py + routes/ + templates/

作用: Web 工作台, 5 个 Tab, 一键暂停 / 一键清仓

3) 分级告警

位置: alerting

作用: /INFO / WARN / CRITICAL / FATAL 各走不同渠道, INFO 聚合避免刷屏

盘中实盘 vs 盘前晨会

维度	盘前晨会 workflow	盘中实盘主循环
触发方式	cron 每天 9:00 跑一次	启动后常驻进程，每分钟一轮
时延要求	30 分钟内出晨报即可	信号产生到下单 ≤ 3 秒
异常容忍	跑挂了人工补一次就行	跑挂了可能在错的时间点重仓
状态	输入输出都是「快照」	必须维护「跨进程共享状态」
UI	一份 .md / .html 晨报	实时 Web 控制台 + 操作面板
失败的代价	当天没晨报	真金白银亏损

工程重点：

- 持续运行怎么保证不挂 => 挂了怎么自愈、什么级别该停手；
- 跨进程状态如何设计 => 让 Web 控制台和主循环不相互干扰；
- 告警怎么分级 => 避免一天 200 条消息，看不过来。

盘中实盘 vs 盘前晨会

Thinking: 这 60 秒里, 程序在干啥?

大部分时间是在 Sleep: 上一轮 run_once() 跑完后, 睡到凑满 60 秒, 再启动下一轮。(若本轮执行本身超过 60 秒, 则紧接着下一轮。)

代码实现中的一轮大致是:

- **可否交易**: 状态若是熔断 / 暂停则直接退出本轮。
- **连通行情**: 连 miniQMT / xtdata, 失败则告警并返回。
- **持仓与盈亏**: 有持仓则用最新价刷新市值与浮动盈亏, 更新当日盈亏, 记在 state 里。
- **熔断**: 当日亏损比例触及阈值 → 设为熔断停牌, 落盘退出。
- **看盘 & 信号**: 对 watch 列表里每一只, 用配置的 策略算买/卖/观望;
- **对已产生的信号逐个处理**: 简化的 单笔规模风控 (如总资金 10%) → dry-run 记一笔模拟单 或 真实下单。
- **落盘**: 把本轮的信号、订单、events、live_state.json 等写回磁盘, 控制台才能读到最新状态。

工程原则

原则1：3 秒原则

从信号产生到下单完成，全链路不能超过 3 秒。

Thinking：为什么是 3 秒？

因为量化策略捕捉的 alpha 通常在 5-30 秒内被市场抹平

你算的指标，别人也在算；你看到的盘口，别人也看得到。

=> 3 秒是 信号-下单 链路的工程上限

环节	时间预算
信号计算（取近 50 根 K 线 + 算 MACD）	≤ 0.5s
风控审批（仓位 / 集中度 / 现金 / 涨跌停）	≤ 0.3s
miniQMT 下单（含挂单回执）	≤ 1.0s
buffer（网络抖动 / GC）	≤ 1.0s

主路径**全部用本地内存 + 同步调用**，不走 HTTP / 数据库 round-trip（请求发出→服务端处理→响应返回的完整往返过程）。
Web 控制台的双向通信走 共享 state 文件，是异步的，不在主路径上。

工程原则

原则2：异常处理金字塔

不要用一刀切的 try/except 把所有错误都吞掉。

不同级别的异常走不同处理路径，自愈的归自愈，要停手的要敢停。

级别	含义	典型场景	处理动作
L1	数据层异常	拉 tick 网络抖动一次	跳过本轮，不告警，下轮继续
L2	风控否决	仓位超限 / 现金不足	不下单，记 INFO
L3	订单失败	miniQMT 临时拒绝（撤单中）	WARN + 重试 1 次
L4	系统级异常	xtdata 连接断 / 行情卡死 5 分钟	CRITICAL + 暂停所有交易
L5	不可恢复	风控引擎本身崩溃	FATAL + 进程退出 + 等人工

低级别自愈，高级别停手，所有级别都留痕（events 列表 + state 落盘，事后可溯）。

=> 这样事后复盘时一眼就能看到「今天下午 14:23 触发了一次 L4，自动停了，不是策略问题」。

工程原则

原则3：心跳监控

Thinking：如何跑了一下午没新事件，可能是什么情况？

可能是没行情，也可能是连接早就断了

=> 区分它们的唯一办法是心跳，所有外部依赖都必须有心跳。

AI量化系统中的三处心跳：

- miniQMT 连接：每 30 秒查账户资产，连续失败即告警；
- 行情源：每分钟拉一根 tick 验证数据通路；
- 主循环：上报最新时间戳，30 分钟无更新则判定线程死锁。

工程原则

原则4：状态分离（UI 与执行解耦）

CEO 控制台（Web）和主循环（live_loop）不能跑在同一个事件循环里

=> 否则 UI 卡了交易也跟着卡。

两者通过一份共享状态文件通信：



- UI 重启不影响交易；
- 主循环挂了 Web 还能看历史 state 复盘；
- CEO 改 state 文件里的控制字段，主循环轮询读取就能执行，不需要使用消息队列。

项目结构

```
+-- CASE-AI量化系统/           Web 工作台 (python app.py)
| +-- app.py                   FastAPI + Tailwind + Alpine + Plotly
| +-- .env.example             (DASHSCOPE / QMT / 钉钉 / MySQL)
| +-- requirements.txt
| +-- README.md
| +-- live_trading/           主循环 + state (核心)
| | +-- live_loop.py          run_once() 9 步主循环
| | +-- state_store.py        JSON 文件 + 原子写
| | +-- miniqmt_trader_v2.py   实盘下单封装
| +-- alerting/
| | +-- alert_router.py        4 级路由 + INFO 聚合
| +-- dragon_strategy/        龙头战法 (CASE-龙头战法 副本)
| | +-- dragon_picker.py
| +-- morning_brief/          21 节晨会工作流 (内嵌副本)
| | +-- graph.py / pusher.py / lib/
| +-- routes/                 REST API (FastAPI)
| | +-- live.py system.py dragon.py morning.py backtest.py
```

项目结构

- | +-- templates/ Jinja2 + Tailwind + Alpine
 - | | +-- base.html live.html system.html chat.html ...
- | +-- static/
- | +-- pages/
 - | | +-- tab1_chat.py Gradio 投研对话 (Charles)
- | +-- lib/ 工作台胶水
 - | | +-- paths.py 路经常量 (全部指向同级目录)
 - | | +-- live_simulator.py 把 LiveTradingLoop 包成后台线程
 - | | +-- strategy_registry.py 策略注册中心
 - | | +-- backtest_data.py engine.py metrics.py
- | +-- config/ 运行时配置 (yaml 热加载)
 - | | +-- mock_positions.yaml 初始持仓 (真实价格驱动的演示组合)
 - | | +-- strategies.yaml 策略路由 (默认 + per_stock)
 - | | +-- watch_pool.yaml 自选监控池
- | +-- outputs/
 - | | +-- live_state.json 共享状态契约
 - | | +-- live_approvals.json
- | +-- data/cache/ 晨会缓存

自动闭环与分级告警

自动闭环

自动闭环：主循环 9 步

#	步骤	在干什么	失败的等级
0	check_control	读 <code>state.trading_status</code> , HALTED / PAUSED 直接跳过本轮	—
1	health_check	miniQMT 连接 + xtda 行情心跳	L4 (CRITICAL)
2	update_positions	拉最新持仓 + 算当日盈亏, 写入 <code>pnl_history</code>	L1
3	check_circuit_breaker	当日累计盈亏是否跌破熔断线 (默认 -2%)	触发即停
4	evaluate_stop_loss	持仓股是否触发个股止损 (默认 -2% / 持有 H 日)	L2
5	evaluate_signals	候选股是否出现新信号 (默认 5 分钟 MACD, 可外部注入策略路由器)	L1
6	risk_check	风控审批 (仓位 / 集中度 / 现金 / 涨跌停)	L2
7	place_orders	dry-run 模拟单 / 真实 miniQMT 下单	L3
8	save_state	落盘 <code>outputs/live_state.json</code> + 滚动 <code>events / signals / orders</code>	—

0 号步骤先于一切。它读的是 CEO 控制台写进来的意图，如果你点了暂停，下一轮就生效，不需要进程重启。

9 步严格顺序，不需要并行。即使是3 秒的预算，并行带来的复杂度远大于收益。

每一步都可以失败。每一步都有自己的异常等级（见原则2），不会一处出错全盘退出。

State 设计

Thinking: State 设计的重要性如何?

State = 跨进程共享的核心契约

整个系统的灵魂在 `live_trading/state_store.py` 维护的JSON: `outputs/live_state.json`。

作用包括:

- 主循环的记忆 (崩了重启可以从这里恢复)
- Web 控制台的数据源 (每 5 秒轮询一次刷新页面)
- 如果你点击「暂停买入」, 会修改这里的 `control.pause_buying`

State 设计

字段全景：

```
{
  "trading_status": "RUNNING",    // RUNNING / PAUSED / HALTED
  "capital":    1000000,
  "positions": [
    { "code": "600519.SH", "name": "贵州茅台", "volume": 100,
      "cost": 1464.49, "cur_price": 1407.20,
      "market_value": 140720, "pnl": -5729, "pnl_pct": -0.039 }
  ],
  "today_pnl":    -3210.5,
  "today_pnl_pct": -0.0032,
  "events": [ /* 滚动 200 条, 含 loop_cycle / 告警 */ ],
  "signals": [ /* 滚动 100 条 */ ],
  "orders": [ /* 滚动 100 条 */ ],
  "pnl_history": [ /* 滚动 500 条, 用于盈亏曲线 */ ],
```

```
"control": {
  "pause_buying": false,    // CEO 写, 主循环读
  "force_clear_all": false,
  "max_daily_loss": -0.02,
  "dry_run":    true
},
"health": {
  "miniqmt_connected": true,
  "last_heartbeat": "2026-04-28T11:23:25",
  "errors_24h":    0
},
"_updated_at": "2026-04-28T11:23:25"
}
```

pnl (Profit and Loss) : 这一笔持仓相对于成本价算出来的浮动盈亏金额

_updated_at: 本条 live_state 最近一次被写回磁盘的时间

outputs/live_state.json

写入的原子性

Thinking: `live_state.json`的写入如何保证原子性?

跨进程最容易出的事故是reader 读到半截的 JSON。

`state_store.save()` 用了简单&容易被忽略的技巧

```
def save(self, state: dict):
    state = {**state, "_updated_at":
datetime.now().isoformat(timespec="seconds")}
    tmp_path = self.state_file.with_suffix(".tmp")
    tmp_path.write_text(json.dumps(state, ensure_ascii=False, indent=2),
                        encoding="utf-8")
    os.replace(tmp_path, self.state_file) # 这一步在所有主流文件系统上
都是原子的
```

先写 `.tmp` 再 `os.replace`: reader 永远读到的是上一份完整的或这一份完整的

`os.replace(tmp, target)` 是原子操作: 它瞬间把目录里的文件名指向 `.tmp` 的内容, 旧文件直接被覆盖。

Thinking: 为什么必须用 `replace` 而不是直接 `write_text` 到目标文件?

直接写入是流式分块的, reader 可能在写到一半时打开文件, 读到半截 JSON。

`replace` 不碰磁盘数据块, 所以不存在中间态——reader 要么读到旧版完整文件, 要么读到新版完整文件

[live_trading/state_store.py](#)

谁读谁写

Thinking: live_state.json中的字段，谁读谁写呢？

字段	主循环	Web 控制台	谁是真相
positions / today_pnl / pnl_history	写	只读	主循环
events / signals / orders	append	只读	主循环
health.*	写	只读	主循环
trading_status	触发熔断时写 HALTED	一键暂停时写 PAUSED	谁先写谁先生效，下一轮自然生效
control.pause_buying / force_clear_all / max_daily_loss / dry_run	只读	写	Web 控制台

设计原则：主循环写事实（已经发生的），Web 写意图（希望发生的）。

=> 不要让 Web 直接改 positions / orders 这些事实字段

=> 交易事实的唯一来源是主循环本身。

主循环代码骨架

```
def run_once(self):
    cycle_start = time.time()
    s = self.state_store.load()
    # 0) 控制状态
    if s.get("trading_status") == "HALTED":
        self.alert.alert("WARN", "交易已熔断, 跳过本轮"); return
    if s.get("trading_status") == "PAUSED":
        self.alert.alert("INFO", "交易已暂停 (CEO 控制台暂停)"); return
    # 1) health -- L4
    try:
        self.market.connect()
        s["health"]["miniqt_connected"] = True
        s["health"]["last_heartbeat"] = datetime.now().isoformat()
    except Exception as e:
        s["health"]["miniqt_connected"] = False
        self.alert.alert("CRITICAL", "miniQMT 连接失败", str(e))
        self.state_store.save(s); return
```

2) 更新持仓 + 当日盈亏

```
positions = update_positions_from_market(s["positions"], self.market)
today_pnl, today_pnl_pct = calc_today_pnl(positions, self.capital)
s["positions"] = positions
s["today_pnl"] = today_pnl
s["today_pnl_pct"] = today_pnl_pct
s["pnl_history"].append({"ts": datetime.now().isoformat(),
                        "pnl": today_pnl, "pnl_pct": today_pnl_pct})
s["pnl_history"] = s["pnl_history"][-500:]
```

3) 熔断 -- 触发即停

```
if today_pnl_pct <= self.max_daily_loss_pct:
    s["trading_status"] = "HALTED"
    self.alert.alert("CRITICAL", "触发当日亏损熔断",
                    f"今日 {today_pnl_pct:.2%} 跌破"
                    {self.max_daily_loss_pct:.2%}")
    self.state_store.save(s); return
```

live_trading/live_loop.py

主循环代码骨架

4-5) 评估信号 (默认 MACD; 注入了 signal_evaluator 走策略路由器)

```
new_signals = []
for code in self.watch_stocks:
    if self.signal_evaluator is not None:
        result = self.signal_evaluator(code, self.market, self.capital)
        if not result or result.get("side") == "hold":
            continue
        sig = {"code": code, "side": result["side"],
              "strategy": result.get("strategy"), "reason": result.get("reason")}
    else:
        df = self.market.get_recent_kline(code, period="5m", count=50)
        side = evaluate_macd_signal(df)
        if side == "hold":
            continue
        sig = {"code": code, "side": side, "strategy": "macd_5min"}
    new_signals.append(sig)
    self.alert.alert("INFO", f"信号触发 -> {sig['side']} {code}")
```

6-7) 风控 + 下单 (异常金字塔在 _handle_signal 里)

```
for sig in new_signals:
    order = self._handle_signal(s, sig)
    self.state_store.append_order(order)
```

8) 落盘

```
s["events"].append({"type": "loop_cycle",
                    "signal_count": len(new_signals),
                    "duration_ms": int((time.time() - cycle_start) * 1000)})
s["events"] = s["events"][-200:]
self.state_store.save(s)
```

主循环代码骨架

_handle_signal() 里设置了异常金字塔:

拦截点	等级	order.status	触发场景
control.pause_buying 拦截	L2 INFO	paused_by_ceo	CEO 在 Web 一键暂停了买入
拿不到价格	L1 静默	rejected	tick 网络抖动, 不告警 , 下轮重试
dry-run 下单	L0 INFO	dry_run	模拟成功
真实下单失败	L3 WARN	failed	miniQMT 临时拒绝
下单异常	L4 CRITICAL	exception	上下游崩了, 下一轮 health 大概率也会挂

信号评估

信号评估 —— 默认 MACD，可注入策略路由

主循环不绑定任何策略。LiveTradingLoop 接受一个可选的 signal_evaluator 回调：

- 没注入：默认对每只 watch 股票拉 50 根 5 分钟 K 线，算 MACD，金叉 → buy，死叉 → sell；
- 注入了：由外部回调决定每只股票走哪个策略 —— 这就是 Web 工作台「策略路由表」干的事。

config/strategies.yaml:

```
default: macd_5min    # 默认策略
per_stock:           # 个股覆盖
  600519.SH: macd_1d   # 茅台用日 K MACD
  513100.SH: dragon_picker # 纳指 ETF 跑龙头逻辑
  002594.SZ: dual_ma_5min
```

策略本身在 lib/strategy_registry.py 注册成 5 大类（MACD / 双均线 / 多因子 Top / 龙头 / 网格），策略路由器读 yaml 决定每只股票走哪个信号评估 signal_evaluator。

[config/strategies.yaml](#)

运行结果

```
python live_trading\live_loop.py --once --stocks 600519.SH,513100.SH,002594.SZ
```

```
xtquant 文档地址: http://dict.thinktrader.net/nativeApi/start\_now.html
```

```
***** xtdata 连接成功 2026-04-28 12:19:22 *****
```

```
服务地址: 127.0.0.1:58610
```

```
数据路径: D:\光大证券金阳光QMT实盘\bin.x64\../userdata_mini/datadir
```

```
[完成] {'action': 'cycle_done', 'duration_ms': 6742, 'new_signals': 0}
```

```
state 落盘: outputs/live_state.json
```

第一次启动 6.7 秒（含 xtdata 握手），后续每轮稳定在 400ms ~ 1.5s。

真实 state 长什么样

Thinking: 真实 state 长什么样?

outputs/live_state.json 保存了真实数据, 包括:

持仓的 cur_price 是 xtdata 拉的当日真实收盘,

pnl_history 是从4月1日建仓以来每分钟一条的浮盈快照, 已经累积了 70+ 个 loop_cycle 事件、21 条真实信号:

```
{  
  "trading_status": "RUNNING",  
  "capital": 1000000,  
  "positions": [  
    {"code": "600519.SH", "name": "贵州茅台", "volume": 100, "cost": 1464.49, "cur_price": 1401.00, "pnl": -6349, "pnl_pct": -0.043},  
    {"code": "688981.SH", "name": "中芯国际", "volume": 1900, "cost": 96.40, "cur_price": 115.80, "pnl": +36860, "pnl_pct": +0.201},  
    {"code": "300750.SZ", "name": "宁德时代", "volume": 400, "cost": 409.73, "cur_price": 428.81, "pnl": +7632, "pnl_pct": +0.047},  
    {"code": "513100.SH", "name": "纳指 ETF", "volume": 110000, "cost": 1.72, "cur_price": 1.956, "pnl": +25960, "pnl_pct": +0.137},  
    {"code": "000001.SZ", "name": "平安银行", "volume": 17100, "cost": 11.09, "cur_price": 11.39, "pnl": +5130, "pnl_pct": +0.027}  
  ],  
}
```

真实 state 长什么样

```
"today_pnl": 69233, // 4-1 建仓至今 +6.92%
"today_pnl_pct": 0.0692,
"signals": [
  /* 21 条真实信号, 比如: */
  {"ts": "2026-04-13", "code": "300750.SZ", "side": "buy", "strategy": "macd_1d",
   "reason": "[日K] 金叉 12/26/9 DIF-DEA=+0.0982"},
  {"ts": "2026-04-08", "code": "513100.SH", "side": "buy", "strategy": "turtle_donchian",
   "reason": "收盘 1.83 突破 20 日高 1.80"},
  {"ts": "2026-04-13", "code": "600519.SH", "side": "sell", "strategy": "boll_revert",
   "reason": "跌破中轨 1443.34 止盈 (前 1453.96/中 1441.86)"}
  // ...
],
"events": [ /* 70+ 条 loop_cycle, 平均 duration_ms ≈ 800 */ ],
"pnl_history": [ /* 75 条, 从 4-1 +0.23% 一路爬到 4-27 收盘 +7.20%, 4-28 微回到 +6.92% */ ],
"control": { "pause_buying": false, "force_clear_all": false, "max_daily_loss": -0.02, "dry_run": true },
"health": { "miniqmt_connected": true, "last_heartbeat": "2026-04-28T12:22:19", "errors_24h": 0 }
}
```

state文件就把整个系统的「事实 + 意图 + 健康度」全记录下来了。没有数据库、没有消息队列，Web 控制台轮询这个 JSON => 渲染整个 dashboard。

打卡：搭建盘中主循环



搭一个7×24 小时能跑、被 Web 看见的实盘骨架：

- 定 state 契约：列出 positions / events / signals / orders / control / health 等字段
- 原子写state契约：先写 .tmp 再 os.replace, reader 永远读到完整 JSON。
- 串行执行 9 步 run_once(): control - health - 更新持仓 - 熔断 - 止损 - 信号 - 风控 - 下单 - 落盘, 顺序同步。
- 异常金字塔：每步配等级 (L1 静默 / L2 不下单 / L3 重试 / L4 停手 / L5 退出) , 全部 append 到 events 留痕。

告警 4 级路由

告警 4 级路由

级别	典型场景	渠道	推送策略
INFO	信号触发 / 仓位变化 / dry-run 下单	控制台 + 钉钉	聚合 30 分钟一批
WARN	风控否决 / 订单失败重试 / 网络抖动	控制台 + 钉钉	实时
CRITICAL	触发熔断 / miniQMT 连接断 / 大额亏损	控制台 + 钉钉 + 企业微信 + 邮件	实时, 钉钉 markdown + @all
FATAL	系统崩溃 / 风控引擎失效 / 进程退出前	全部渠道	实时, 失败重试 3 次

INFO 聚合:

- 30 分钟内 5 条 INFO 推一条「过去 30 分钟事件汇总 (5 条)」, 避免刷屏;
- 这样真正的 CRITICAL 就不会被淹没;
- 程序退出前主动 flush_now() 把残余 INFO 推一条收尾。

告警 4 级路由

用法 (被主循环 import)

```
from alerting.alert_router import AlertRouter

router = AlertRouter(info_aggregate_seconds=1800) # 30 分钟聚合一次
router.alert("INFO", "信号触发", "MACD 金叉 -> BUY 600519")
router.alert("WARN", "风控否决", "600519 单笔超 10% 资金上限")
router.alert("CRITICAL", "miniQMT 连接失败", message=str(e))

router.flush_now() # 强制把 INFO 队列推一次
router.shutdown() # 程序退出前 flush + 停聚合线程
```

告警 4 级路由

运行结果: python alerting/alert_router.py

--- 模拟一段盘中事件 (8 条不同级别) ---

[INFO] 11:55:48 @zoe : Zoe 信号触发 / MACD 金叉 -> BUY 600519.SH

-> {'console': True}

[INFO] 11:55:48 @trader: 持仓更新 / 买入 100 股 600519.SH @ 1407.24

-> {'console': True}

[WARN] 11:55:49 @kris : 风控降仓 / Kris 触发 ATR 仓位上限, 建议降至 70%

-> {'console': True, 'dingtalk': False}

[INFO] 11:55:49 @zoe : 信号触发 / MACD 金叉 -> BUY 002594.SZ

-> {'console': True}

[CRIT] 11:55:50 @kris : 单日亏损告警 / 今日累计亏损 -1.8%, 接近熔断线 -2.0%

-> {'console': True, 'dingtalk': False, 'wecom': False, 'email': False}

[INFO] 11:55:50 @zoe : 信号触发 / RSI 超卖 -> BUY 513100.SH

-> {'console': True}

[FATAL] 11:55:51 @trader: miniQMT 连接断开 / 心跳失败 5 次, 已尝试重连 3 次均失败

-> {'console': True, 'dingtalk': False, 'wecom': False, 'email': False}

[INFO] 11:56:00 @system: 策略状态 / 暂停所有买入, 等待人工恢复

--- 等待 5 秒, 看 INFO 聚合推送 ---

[统计]

INFO : 5 条

WARN : 1 条

CRITICAL : 1 条

FATAL : 1 条

[OK] Demo 完成

定时调度

定时调度：让主循环按交易时段自动启停

主循环本身不挑时间，启动后就每分钟巡检一次。实盘只在 9:30–15:00 有意义，不让它 7×24 空转。

时间（周一至周五）	Job	干什么
08:30	数据增量	调 <code>run_daily.py</code> —— 拉昨日 K 线 + 板块指数到 <code>trade_*_daily</code>
09:30	启动主循环	给 <code>LiveSimRunner.start()</code> 发启动信号，主循环开始每分钟巡检
14:55	停止主循环	收盘前 5 分钟主动停，避免空转

定时调度

启动方法

模拟模式: 立刻把 3 个 job 各跑一次, 验证流程通 (推荐第一次先跑这个)

```
python scheduler.py --simulate
```

正式: 阻塞前台运行, Ctrl+C 退出

```
python scheduler.py
```

Thinking: 为什么调度scheduler.py 和 web 分两个进程?

Web (app.py) 和 调度 (scheduler.py) 解耦:

浏览器关了不影响调度, 调度挂了也不影响 Web。

⇒这是工程最佳实践 —— 生命周期不一样的东西, 不要绑在一起。

Web 跟着用户开关, 调度 7×24 常驻。

如何上生产

上生产 = systemd / 任务计划程序 包一层

scheduler.py 是阻塞前台进程，如果要做到“开机自启 + 挂了自动重启”，需要在外面包一层：

- Linux：写一个 systemd unit, Restart=on-failure;
- Windows：「任务计划程序」绑「计算机启动时」+「失败时重新启动」。

scheduler.py 是阻塞前台进程，不具备后台守护和自恢复能力。

systemd (Linux) 和任务计划程序 (Windows) 是系统级进程管家，把它们包一层后：开机自动拉起程序，进程崩溃或退出时自动重启 (Restart=on-failure) => 实现生产级持续运行。

如何上生产

上生产前的 checklist

- 配齐 .env: QMT_PATH / ACCOUNT_ID / DINGTALK_WEBHOOK / SMTP_*;
- 主循环用 systemd (Linux) / 任务计划程序 (Windows) 包成开机自启;
- state_file 放 SSD —— 每分钟一次 fsync, HDD 长期会拖慢;
- dry-run 跑一周, 看 events / orders / pnl_history 是否合理;
- 切实盘前先用纳指 ETF (513100, 单价约 1.89 元) 小额试水, 一手不到 200 块。

Summary

- 3 秒原则是核心约束。所有设计都要服从它 —— 主路径只走本地内存 + 同步调用。
- 异常金字塔 > 全 try/except。

L1 静默 / L2 不下单 / L3 重试 / L4 停手 / L5 退出，自愈和上报分两条路径。

- state 文件 = 跨进程通信契约。先写 .tmp 再 os.replace，原子又简单。
- 主循环写事实，Web 写意图 => 不要让 Web 改 positions / orders。
- 告警的灵魂是 INFO 聚合。不聚合，CRITICAL 会被淹没。
- Web 和 调度 分进程。生命周期不一样的东西不要绑在一起。
- dry-run 是安全锁。可以先跑一周，看 state 里的数字是否符合预期。

打卡：分级告警 + 自动启停



以「INFO 30 分钟聚合 + 9:30 启 / 14:55 停」为例，让系统自己上下班且不刷屏：

- 定 4 级语义：INFO（信号 / dry-run）、WARN（风控 / 重试）、CRITICAL（熔断 / 断连）、FATAL（崩溃）。
- AlertRouter：每级路由对应渠道（控制台 / 钉钉 / 企微 / 邮件）；INFO 走聚合队列，其他级别立即推。
- 主循环接入：所有事件统一调 `router.alert(level, ...)`，进程退出前 `flush_now()`。
- 调度独立进程：APScheduler 注 3 个 cron：08:30 数据增量、09:30 启主循环、14:55 停。
- 外包一层重启：Linux 用 systemd，Windows 用任务计划「开机启动 + 失败重启」。

CEO控制台

CEO控制台

CEO 控制台：

打造一个能跑的 Web 工作台，主循环在跑，CEO 需要：

- 看到当前持仓 / 当日盈亏 / 信号订单面板；
- 看到事件流（按 INFO / WARN / CRITICAL 上色徽章）；
- 不对劲时一键暂停 / 一键清仓（带二次确认，避免误点）；
- 调整熔断阈值 / dry-run 开关；
- 顺便能看晨会、能跟 Charles 对话做投研、能发起单股回测。

技术选型

选型对比

方案	优点	缺点	本系统
FastAPI + Tailwind + Alpine + Gradio	单进程；无前端构建；能同时挂 Gradio	不适合复杂前端交互	主形态
Gradio 单文件	最快上手，对话式交互天然适合	整站 Gradio 不够灵活	投研对话子 Tab 用
React/Vue + 独立 API	工业级 UI	项目过重，要搭构建流水线	不用

投研对话恰好是 Gradio 最擅长的对话式交互；

其它 Tab 用 FastAPI 模板就够。

=> 把 Gradio 通过 `gr.mount_gradio_app()` 挂到 `/gradio-chat/`，再用 `iframe` 嵌进 `/chat` 页面

=> 一个进程，两套 UI 共存。

技术选型

- FastAPI (后端框架)

Python 写路由 / REST API / 模板渲染。

routes/{live,system,dragon,morning,backtest}.py 提供数据接口;

用 Jinja2 渲染 templates/*.html 出页面;

同时支持挂载 Gradio 子应用, 所以一个进程能既跑 REST 又跑 Gradio

- Tailwind CSS (样式工具集)

用 class 直接写样式 (如 text-xs text-gray-500) , 不写一行 CSS。

CDN 引入, 无构建步骤。整套 dashboard 的卡片、表格、徽章、按钮都使用它

技术选型

- Alpine.js (轻量前端交互)

仅 14KB 的轻量前端框架，采用类似 React 的声明式思路，通过 x-data / x-show / x-for 等指令直接将数据绑定到 DOM。

这里用它实现每 5 秒轮询 state、按钮触发后端、弹窗确认等交互

- Gradio (对话式 UI)

用于 /chat 投研对话。Charles 的流式回复、思考过程折叠、推荐对话列表

=> Gradio 的 Chatbot 组件天然支持，自己写要几百行

因为 dashboard 80% 是表格 + 数字 + 按钮 => Alpine 完全够；

剩下 20% 是对话 => Gradio 效率更高

控制台Tab

控制台 5 个 Tab

1) 实盘监控

- 路径: /live/sim、 /live/real
- 持仓 / 盈亏曲线 / 信号订单 / 龙头候选面板 / 事件流告警徽章 / 一键操作 (含二次确认)

2) 回测

- 路径: /backtest
- 单股策略历史回测, 支持 MySQL wucai_trade.* + xtdata 双数据源

控制台Tab

3) 投研对话

- 路径: /chat
- 调 nanobot 框架的 Charles agent

4) 晨会分析

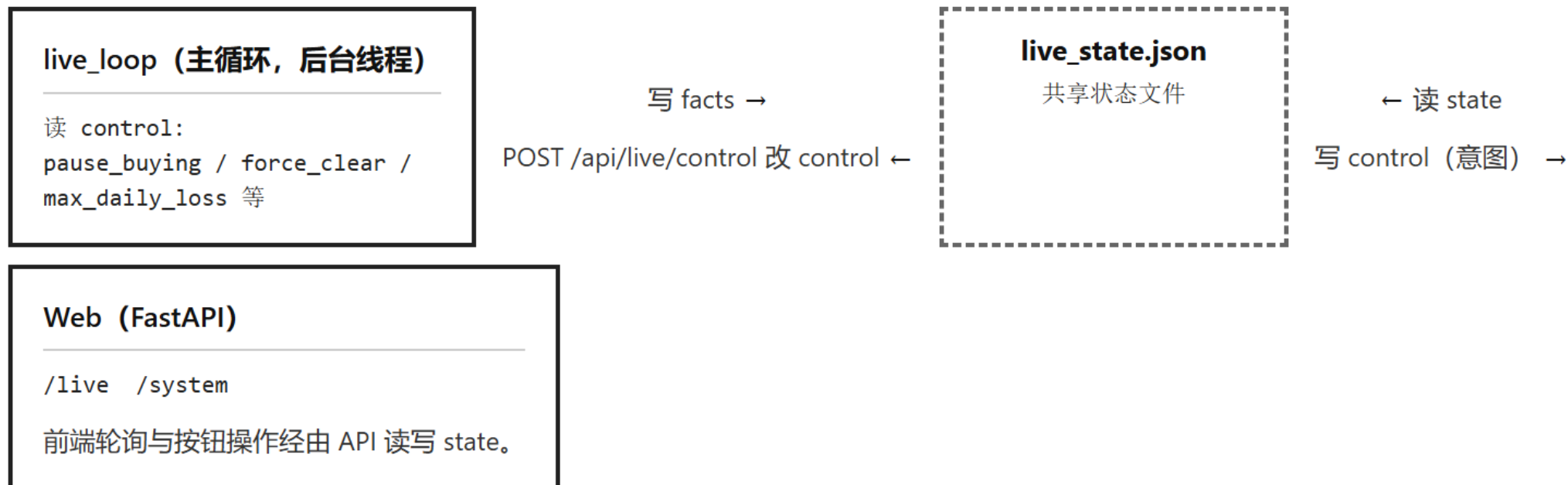
- 路径: /morning
- 晨会 workflow: 板块强度 + Top 板块多因子选股 + 推送

5) 系统状态

- 路径: /system
- xtdata / DASHSCOPE / live_state / 晨会 HTML 等组件健康检查

双向通信原理

双向通信原理



主循环写事实，Web 写意图：Web 永远不能直接改 positions / orders 这种事实字段。

运行结果

python app.py

-> http://localhost:7865 (默认进 /live/sim)

AI 量化系统 | CEO 控制台

实盘监控

回测

投研对话

晨会分析

系统状态

模拟盘

实盘

模拟盘 (/live/sim): 自动跑策略, 不会真实下单

实时持仓

5 只持仓 · 3 只待入场 · 点策略名可切换

+ 添加股票

代码	名称	执行策略	持仓	成本	现价	市值	浮盈/亏	盈亏%
600519.SH	贵州茅台	布林带均值回归 (20-2σ)	100	1,464.49	1,405.46	140,546	-5903	-4.03%
688981.SH	中芯国际	龙头首板战法 (5 法则简化)	1,900	96.40	110.00	209,000	+25840	14.11%
300750.SZ	宁德时代	MACD-日K线 (12/26/9-波段)	400	409.73	429.82	171,928	+8036	4.90%
513100.SH	纳指ETF	海龟唐奇安通道 (20入/10出)	110,000	1.72	1.95	214,830	+25630	13.55%
000001.SZ	平安银行	布林带均值回归 (20-2σ)	17,100	11.09	11.51	196,821	+7182	3.79%
002241.SZ	待入场	龙头首板战法 (5 法则简化)	0	-	-	-	-	-
300059.SZ	待入场	龙头首板战法 (5 法则简化)	0	-	-	-	-	-
300014.SZ	待入场	龙头首板战法 (5 法则简化)	0	-	-	-	-	-

资金面板

RUNNING

停止

?

总资产

1,060,785

+60785 (6.08%) vs 初始 1,000,000

持仓市值

933,125

占 88.0% · 5 只

可用现金

127,660

占 12.0% · 可加仓

监控 8 只 · 已跑 1 轮 · 10:06:01

重置持仓 清空历史

更新于 10:06:21

成交流水

左下方「信号」触发后, 主循环按风控算股数 → 这里是成交记录⁰条

录

例: 600519 buy 100 股 @ 1407.24 (dry-run, 不真下单)

暂无成交

下一笔买/卖信号触发后会写入这里

运行结果

启动后看到的浮盈面板，数据来源是 config/mock_positions.yaml:

```
# 100 万本金, 5 只票各分配 ~19 万, 按 2026-04-01 真实开盘价等权建仓
capital: 1000000
positions:
  - {code: 600519.SH, name: 贵州茅台, volume: 100, cost: 1464.49} # 14.6 万
  - {code: 688981.SH, name: 中芯国际, volume: 1900, cost: 96.40} # 18.3 万
  - {code: 300750.SZ, name: 宁德时代, volume: 400, cost: 409.73} # 16.4 万
  - {code: 513100.SH, name: 纳指ETF, volume: 110000, cost: 1.72} # 18.9 万
  - {code: 000001.SZ, name: 平安银行, volume: 17100, cost: 11.09} # 19.0 万
```

运行结果

用国泰君安五步法的框架，简要分析贵州茅台 (600519)

贵州茅台 (600519) 国泰君安五步法简析

一、信息差

市场普遍知晓茅台"高毛利、强品牌"，但易忽略三点新事实：

- ① 2026Q1 营收同比转正 (+0.28%)，终结连续三季下滑，主因 i 茅台单季 215.5 亿 (+32%) ...
- ② 2025 全年归母净利 646 亿 (+6.25%)，经营性现金流 382 亿虽 -14%，但系加大直营备货...
- ③ 渠道结构质变：经销商较 24 年末净减 255 家，飞天批价稳守 2600 以上...

二、逻辑差

旧逻辑 ("提价+批价驱动") 已让位于新逻辑 ("控盘+结构+效率驱动") ...

三、预期差

当前股价隐含悲观预期：市场仍担忧消费疲软拖累高端白酒，忽视茅台"必需奢侈品"属性...

Q1 业绩转正未引发明显估值修复，显示主流资金尚未形成"经营拐点"共识，存在显著预期差。

四、催化剂

短期：2026 年中期分红预案 (预计 6 月公告，股息率或达 3.2%+)

中期：i 茅台上线"智能配额+会员分级"新系统 (2026 年 5 月内测)

长期："茅台国际化"加速 + 数据资产入表试点

运行结果

五、结论与风险闭环

✅ 结论：茅台正处于"新旧逻辑切换完成、业绩拐点确认、估值尚未重估"的黄金窗口期...

⚠️ 风险提示：① 消费复苏不及预期 ② i 茅台增长见顶 ③ 行业政策变动
④ 高估值波动 (PE TTM $\approx 32x$)

> 一句话总结：不是"是否持有"，而是"以何种心态持有"——它已从周期股蜕变为
> 具备永续现金流特征的核心资产，适合长期底仓配置，但需容忍阶段性估值波动。

nanobot 的设计是：让 agent 自己决定要什么数据

=> 同一个问题在不同的工具可用性下走出来的链条天差地别。

CASE: 龙头战法

CASE: 龙头战法

龙头战法理论背景

- Ross Cameron:

美股日内交易 (Day Trading) 代表人物, YouTube 拥有超百万粉丝。

2017 年, 他发起 \$583 挑战: 以 583.15 美元小账户实盘直播, 每日仅交易 1 小时, 平均持仓约 7 分钟, 历经 553 个交易日 (约两年半) 累计盈利突破 100 万美元

每月公开券商对账单接受第三方审计, 被视作小资金日内交易的标杆案例。

CASE: 龙头战法

- Gap and Go:

Ross Cameron (Warrior Trading) 的核心日内交易策略，专做开盘跳空后的顺势动能交易。

核心逻辑：股票因隔夜消息（如财报、公告）导致开盘价较前一日收盘价跳空上涨（通常筛选 $>4\%$ 的缺口），开盘后利用市场惯性继续冲高，交易者顺势入场赚开盘初期的动能波段。

选股与执行：盘前扫描跳空股，确认新闻催化剂，锁定低流通盘、高成交量的小盘股（通常 $< \$20$ ）。

开盘后等待首次回调或整固，出现技术信号（如突破盘前高点、牛旗整理）即入场，持仓通常仅几分钟。

风控与止盈：

不追求一次性赚取巨额暴利，每股盈利 $\$0.3-\0.8 即离场；

止损设得极紧（常低于入场价 $\$0.10$ ），确保单笔亏损可控

CASE：龙头战法

美股 vs A 股：不能照搬

维度	美股（Ross 原版）	A 股（我们的实现）
涨幅限制	无	±10% 涨跌停（北交所 ±30%）
T+0 / T+1	T+0，当日可平	T+1，次日才能卖
挑标的的涨幅阈值	> 10%	> 5%（10% 已涨停，买不到）
价格区间	< \$20	< 30 元
最大风险点	追涨被砸	追涨被涨停板打开放量出货

A 股最致命的差异是 T+1

=> 这就让日内动量策略整个失效

=> 必须改成「今天买，T+1 ~ T+H 卖」的次日跟进型。

龙头战法 V1版本

五大筛选法则

法则	阈值	金融含义
1	当日涨幅 > 5%	动量效应 (Jegadeesh-Titman 1993)
2	涨幅榜前 50	资金共识 + 流动性
3	流通市值 50–200 亿 (30–500 亿可接受)	太大涨不动，太小被庄家拉
4	量比 > 2	量价配合，有量才是真涨
5	排除 ST / 退市 / 价格 > 30 元	黑天鹅规避 + 弹性约束

龙头战法 V1版本

入场出场规则

- 只赚小额确定性利润：每股 0.3–0.8 元就走；
- 盈亏比 $\geq 2:1$ ：胜率 33% 也能盈利；
- 单笔风险 $\leq 1\%$ ：最大亏损 = 总资金 $\times 1\%$ ；
- 时间止损：A 股 T+1 $\rightarrow$ 改为最长持仓 H 日（默认 H=3）；
- 当日累亏 2% 强制收手，不报复性交易。

龙头战法 V2版本

Thinking: 龙头战法 V1版本的最大问题是什么?

v1 只看个股快照。最容易吃亏的场景：一只股涨 7%，但所属板块当日跌。

=> 多半是诱多 —— 主力借板块弱势低位吸筹拉抬，吸引跟风后再砸盘出货。

v2 把之前的 trade_sector_daily 接进来，把选股器picker从个股快照升级到板块视角的龙头判定

v1 dragon_score = f(涨幅, 量比, 流通市值, 涨幅榜排名, 价格)

v2 dragon_score = v1_score

+ sector_resonance_score(板块涨幅, 板块上涨家数占比)

v2 硬过滤补丁:

- 涨幅 $\geq 9.5\%$ 不收 (T+1 高开会让回测虚胖)
- 上市天数 $< 60d$ 不收 (次新股形态不可信)
- 板块当日涨幅 $< 0.5\%$ 或 上涨家数占比 $< 40\%$ 不收 (孤雁淘汰)

板块口径：这里用申万二级 (sector_2)

trade_sector_daily 之前已经创建过，可以直接使用

龙头战法 V2版本

dragon_picker.py 里 v2 的核心就这几行:

```
SECTOR_MIN_CHANGE_PCT = 0.005 # 板块当日涨幅 >= 0.5%
SECTOR_MIN_RISE_RATIO = 0.40 # 板块上涨家数占比 >= 40%

def passes_sector_resonance(stock_data: dict) -> bool:
    s_chg = stock_data.get("sector_change_pct")
    s_rise = stock_data.get("sector_rise_ratio")
    if s_chg is None or s_rise is None:
        return False
    return s_chg >= SECTOR_MIN_CHANGE_PCT and s_rise >= SECTOR_MIN_RISE_RATIO
```

思路: 板块共振

用交易日里 板块指数的涨跌 (change_pct) 和 板块内上涨家数占比 (rise_ratio = rise_count/stock_count), 判断一只股票是不是跟在一起走强的板块里。

回测里 dragon_backtest.load_sector_panel() 一次性把 trade_sector_daily 的 change_pct / rise_count / stock_count 拉到内存, 然后按 (sector_2, trade_date) join 给每只候选股。

默认 v2 全开, 设置参数 --no-sector-resonance 可以关掉这条硬过滤

三层递进（选股picker）

三层递进：选股 picker → 回测 backtest → 复盘 replay

=> 策略验证 先看逻辑通不通，再看历史统计如何，再挑一只复盘 为什么是它。

- 第一层：选股 picker（5 秒看清逻辑）

`python dragon_strategy\dragon_picker.py`

用 12 只构造样本（覆盖各种典型场景），组合包含：

4 只 v2 通过的「真龙」（板块强 + 涨幅适中 + 30-500 亿）；

3 只 v1 能进 v2 拦的「诱多」（接近涨停 / 板块没共振 / 次新股）；

5 只 v1 直接淘汰的（大盘股 / 价格高 / 量比小 / ST）。

三层递进 (选股picker)

python dragon_strategy/dragon_picker.py

[3] 筛选后龙头候选 4 只 (按 dragon_score 排序)

[4] 688981.SH 中芯国际 涨幅 +7.20% 量比 4.8 市值 320 亿 价 28.60

板块 [半导体] +3.20% 涨家占比 70% -> dragon_score +4.88

[10] 002241.SZ 歌尔股份 涨幅 +5.80% 量比 3.8 市值 150 亿 价 21.30

板块 [消费电子] +2.20% 涨家占比 62% -> dragon_score +4.02

[8] 300059.SZ 东方财富 涨幅 +6.30% 量比 3.2 市值 180 亿 价 18.60

板块 [证券] +1.80% 涨家占比 58% -> dragon_score +3.99

[7] 300014.SZ 亿纬锂能 涨幅 +6.80% 量比 3.6 市值 280 亿 价 25.80

板块 [电池] +2.80% 涨家占比 65% -> dragon_score +3.90

[4] Top 3 入场参数 (Base Hit 小赢 + 2:1 盈亏比 + 1% 单笔风险)

688981.SH 中芯国际 入场 28.60 止损 28.03 (-2.0%) 目标 29.74 (+4.0%)

1700 股 = 48,620 元 最大亏 972 / 最大盈 1,945 元

12 只里 4 只通过, 剩下 8 只被各种法则拦掉:

卫宁健康 (涨 8.5%) → v2-8 板块共振失败 (医疗信息化 +0.3% < 0.5%)

华友钴业 (涨 9.7%) → v2-6 接近涨停板 ($\geq 9.5\%$)

次新示例 (上市 30 天) → v2-7 次新股 (< 60 个交易日)

宁德时代 (市值 1250 亿) → v1-3 流通市值 > 500 亿

赣锋锂业 (价 32.1) → v1 价格上限 30 元

通威股份 (量比 1.6) → v1-4 量比 < 2.0

*ST 测试 → v1-5 ST

茅台 (涨 1.8%) → v1-1 涨幅 < 5%

5 秒看清: v1 漏了什么 / v2 多拦了什么。

这是 12 只构造样本 (覆盖典型场景), 不是真实当日数据, 但每一条都对应真实场景。

三层递进（回测backtest）

- 第二层：全市场历史回测（10 秒看清取舍）

v1 对照: 关共振 + 不限涨停 + 不限次新

```
python dragon_strategy\dragon_backtest.py --start 2025-01-01 --end 2025-06-30 `
    --top 5 --hold 3 --no-sector-resonance --max-change 0.10 --min-listed-days 0
```

v2 默认: 共振 + 排除涨停 + 排除次新

```
python dragon_strategy\dragon_backtest.py --start 2025-01-01 --end 2025-06-30 --top 5 --hold 3
```

回测口径（A 股 T+1 友好）：

- 每个交易日 T 跑选股器picker 拿 Top 5 候选；
- 模拟 T+1 开盘买入、T+H 收盘卖出（ $H \in \{1, 3, 5\}$ ）；
- 汇总：胜率 / 平均收益 / 等权累计净值 / 年化 / Sharpe / 最大回撤；
- 按 sector_2 分组，看龙头是否集中在少数题材；
- CSV 落 outputs/，每行交易都带 sector_chg / sector_rise，方便复盘。

三层递进（回测backtest）

真实回测对照（2025-01-01 ~ 2025-06-30, Top 5 / hold 3, 本地 wucai_trade 786,922 行日 K + 15,210 行板块面板）：

口径	信号笔数	胜率	平均收益	累计	年化	Sharpe	MDD
v1（关共振 / 不限涨停 / 不限次新）	77	54.5%	+1.30%	+12.6%	+231%	1.94	-18.5%
v2（默认全开）	6	83.3%	+5.10%	+0.14%	+18.9%	0.72	-6.84%

Thinking: 不是简单的v2 更好吗？

v2 把胜率从 54% 拉到 83%、单笔均收从 +1.3% 拉到 +5.1%、MDD 从 -18% 收到 -6.8%

=> 共振 + 硬规则带来的质量提升；

但 v2 信号数从 77 暴跌到 6（共振太严），导致累计净值 v1 反而更高（频次优势）；

Thinking: 少而精 vs 多而杂 => 没有免费午餐。

三层递进（回测backtest）

按 sector_2 看板块集中度（同一组回测的副产物，CSV 里能直接看）：

dragon_backtest.py 运行时的终端汇总（或与 CSV 对照）

v1 信号 77 笔的板块分布 (Top 10):

种植业	n=5	胜率 100%	均收 +6.93%
电力	n=4	胜率 75%	均收 +1.56%
IT 服务	n=4	胜率 25%	均收 -4.90%
化学制药	n=4	胜率 75%	均收 +1.87%
农化制品	n=3	胜率 67%	均收 -6.21%
汽车零部件	n=3	胜率 67%	均收 +0.74%
房地产开发	n=3	胜率 67%	均收 +3.77%
一般零售	n=3	胜率 100%	均收 +16.56%

...

v2 信号 6 笔的板块分布:

种植业	n=4	胜率 100%	均收 +7.19%	<- 4 笔都集中在这一个板块!
养殖业	n=1	胜率 100%	均收 +8.70%	
燃气	n=1	胜率 0%	均收 -6.84%	

- v2 把 6 个信号里的 4 个押在种植业

=> 共振筛选的副作用是「找到强共振的当天，板块往往就 1-2 个，候选高度集中」。值得留意的风险点：v2 的低 MDD 部分是板块押对了，不全是策略本身硬。

- v1 那 77 笔里，「IT 服务 n=4 胜率 25%」「农化制品 n=3 均收 -6.21%」就是 v2 真正想拦的弱共振假信号

=> 板块本身在跌，里面跳出来的强势股大概率是诱多。

三层递进（复盘replay）

- 第三层：针对单个标的的复盘

同时跑 v1/v2，看看某只股在某天，选股器picker 是怎么决策的

东山精密 2025-09 ~ 2025-12 (放宽价格/市值上限, 因为它当时已经长成大票)

```
python dragon_strategy\dragon_replay_one.py --code 002384.SZ `
```

```
--start 2025-09-01 --end 2025-12-31 --hold 3 `
```

```
--max-price 200 --mcap-high 5000e8
```

日期	v1 触发原因	v2 拦掉原因	真实未来 3 天
2025-11-06	涨 +10% 量比 2.0	v2-6: 涨幅 10% 是涨停 (T+1 高开污染统计)	+2.13%
2025-12-12	涨 +7.75% 量比 2.1	v2-8: 板块只 -0.25% (共振失败)	+0.51%

v2 拦掉的虽然没大亏，但收益都明显低于 v2 真正放行的「板块共振 + 涨幅适中」的票。

少 ≠ 错，少 = 等到了真的

龙头候选面板

Web 工作台里的龙头候选面板：/live 页面右栏「龙头战法候选」面板，是选股器picker的 Web 入口

=> 默认拉最近一个真实交易日的全市场，按 dragon_score 排序展示。

Thinking: 它的工作流是怎样的？ (4 步串起来)

Step1, 刷新候选 → 后端 _candidates_from_mysql_latest_day() 从 wucaitrade.* 拉 MAX(trade_date) 当天的全市场涨跌 + 板块共振

→ 调 filter_dragon_candidates(v1+v2) → 表格展示

Step2, 用户看到候选名单，决定要不要建仓

Step3, 一键加入监控 → 后端 /api/dragon/bind 把这些代码写进 config/watch_pool.yaml, 同时把 config/strategies.yaml::per_stock 里这些代码全部强制绑成 dragon_picker 策略

Step4, 主循环下一轮 ($\leq 60s$) 热加载 yaml, 自动对这些股票用 strat_dragon 算入场 / 出场信号

dragon_picker 策略 = 选股 + 战法

Thinking: 龙头战法只是选股吗?

不仅是选股, `lib/strategy_registry.py::strat_dragon` 是个完整的 evaluator, 入场和出场都在里面:

- 入场 (buy)

当日涨幅 > 5% 且 量比 > 2 且 价位 < 30 元 且 龙头分 ≥ 1.5

- 出场 (sell)

当日跌幅 > 3% 或 从当日最高点回撤 > 3%

dragon_picker 策略 = 选股 + 战法

运行结果：

启动 python app.py → /live 右栏点「刷新候选」（默认 auto，走 mysql 路径），输出：

trade_date = 2026-04-23 共振=开 候选 2 只

600559.SH 老白干酒 涨 +6.20% 量比 5.0 价 14.21 板块[白酒] +3.05% 分 4.82

601997.SH ... 涨 +7.67% 量比 8.8 价 6.32 板块[城商行] +1.83% 分 4.50

全市场 5000+ 只股票，v2 共振 + 3 条硬规则筛完只剩 2 只。

=> 这是 v2 的质量优先取舍，虽然候选少，但每只都站在强势板块 + 涨幅适中 + 不是次新 + 不是涨停板。

与 v1 对照：同一交易日把共振和涨停 / 次新过滤都关掉，候选 10 只

=> 多出来的 8 只里，大概率混着孤雁和涨停板买不到 => 这是 v2 想拦的。

Summary

Ross 案例对龙头战法的警示:

Ross Cameron 的实盘盈利经第三方审计为真, 但 FTC 调查发现: 复制他策略的跟单者几乎全部亏损, 最终 Warrior Trading 以 300 万美元和解。

这与龙头战法一模一样: 游资大佬的交割单真实, 但散户照做却大亏。

差距不在策略, 而在三个“非人”要求:

- **反人性的止盈:** Ross 的Base Hit赚 3% 就跑, 龙头战法要求次日不涨停就走。

=> 看到股价飙涨, 谁舍得卖? 90% 的人想再等等, 最终盈利变亏损。

- **极窄的容错:** Ross 每笔风险严格 1%, 错十次才伤一成。

=> 散户做龙头往往满仓单吊, 错一次直接腰斩, 没有翻身筹码。

- **机器般的止损:** Ross 止损像程序般果断。

=> 散户看到龙头回调, 总幻想主力洗盘, 该砍时不砍, 小亏拖成深套。

Summary

AI量化建议

- 不要直接复制实盘，至少 dry-run 跑一段时间；
- 用小本金试水（1 万元起步）；
- 记录所有交易，这里的 live_state + 归因看自己是否在执行计划；
- 盈亏比是数学保护，但同样需要纪律支撑。

打卡：CEO工作台



把主循环做成看得见控得住的工作台：

- 5 Tab 骨架： /live /backtest /chat /morning /system, Tailwind + Alpine, 每 5 秒轮询 state。
- 双向通信： UI 只读事实； 暂停 / 清仓 / 改熔断走 POST /api/live/control, 主循环下一轮生效。
- 挂 Gradio /chat: gr.mount_gradio_app() 挂 Charles, iframe 嵌入； nanobot 用后台 loop + queue 桥流式。
- 龙头 picker: 实现 v1 五法则 + v2 板块共振、排除涨停 / 次新, 输出 dragon_score。



Thank You
Using data to solve problems

